

Embeddings, Vector Search, and Hybrid Retrieval

Embeddings place inputs in a model-dependent vector space; similarity is useful only when that geometry matches the task. Retrieval quality depends on candidate coverage, filters, indexing, ranking, and evaluation. Compare with lexical and exact-match baselines, especially for identifiers and specialized vocabulary.

For: Engineers choosing a retrieval approach for search or RAG who need to explain vector similarity, approximate indexes, filters, reranking, and migration rather than assume one database solves relevance.

Suggested pace: 2-3 weeks with a labeled query set | Basic vectors and retrieval concepts

What you will be able to do

- Explain cosine, dot-product, and distance semantics under normalization.
- Distinguish exact nearest-neighbor search from approximate index trade-offs.
- Combine lexical and vector candidates without mixing incompatible raw scores blindly.
- Plan embedding/index version migration and evaluate recall, latency, and cost.

Build the right mental model

Similarity is not probability

A cosine score reflects vector direction, not a calibrated probability that two statements mean the same thing. Different embedding models define different spaces, even with equal dimension counts. Version the model and do not compare incompatible embeddings directly.

Approximation trades resources for recall

Approximate nearest-neighbor structures reduce search work while potentially missing exact neighbors. Parameters affect recall, latency, memory, update cost, and rebuild behavior. Measure under realistic filters and data distribution, not only an unfiltered benchmark.

Hybrid retrieval combines strengths

Lexical search helps exact names, codes, and rare terms; dense retrieval can help semantic matching. Rank fusion combines ordered candidates without assuming raw score scales match. Reranking is another evaluated stage with its own latency and authorization requirements.

Study with evidence.

Build a small dataset with queries requiring exact matches, paraphrases, and filtered access. Label relevant evidence and compare retrieval methods on the same set. Include negative and ambiguous queries so every result is not assumed useful.

Sessions 1-3: Vector semantics

Compute cosine and dot products on small vectors, inspect normalization, and study embedding model inputs and limits. Define what similarity should mean for your application instead of treating all semantic relationships as equivalent.

Deliverable and check: A tiny hand-checked example and an explicit embedding/version contract.

Sessions 4-6: Exact and approximate search

Use a maintained library or database to compare exact retrieval with an approximate index. Measure recall at k, latency, memory, and build/update costs. Include selective metadata filters.

Deliverable and check: A benchmark with workload, parameter settings, and quality trade-offs documented.

Sessions 7-10: Hybrid and reranking

Add lexical candidates, deduplicate IDs, use a justified fusion method, and evaluate optional reranking. Apply authorization before candidates reach external or model-based ranking components.

Deliverable and check: A per-query report explaining why exact identifiers and paraphrases benefit from different signals.

Sessions 11-14: Migration and operations

Create a parallel index for a new embedding model, compare quality, and plan traffic shift and rollback. Track stale versions, deletions, rebuild capacity, and tenant isolation.

Deliverable and check: A migration plan that never silently mixes incompatible vector spaces.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Recover an exact identifier missed by dense search

Users search a knowledge base for error code E417 and for paraphrases of 'connection pool exhausted.' Dense retrieval handles paraphrases reasonably but sometimes misses the exact code or returns a semantically similar unrelated error.

A worked approach

1. Label relevant documents for both query types and run lexical and dense baselines separately. Inspect whether the code was preserved during parsing and tokenization; retrieval cannot recover a field removed at ingestion.
2. Combine bounded candidate sets using a documented rank-fusion method, deduplicate source IDs, and apply filters. Do not add raw lexical and vector scores without a calibrated reason, since their scales have different meanings.
3. Evaluate recall and final ranking for identifier, paraphrase, mixed, and no-answer queries. Add reranking only if its measured quality gain justifies latency and cost, and preserve a fallback if that stage is unavailable.

Expected result

Hybrid retrieval addresses different query signals without claiming that vectors replace exact search. The improvement is demonstrated on a representative labeled set, including cases where neither method finds valid evidence.

Verify the result

- Are identifiers preserved and searchable as exact fields or tokens?
- Are relevance labels independent of the retrieval method being tested?
- Do filters and reranking preserve tenant and document permissions?

Extend the exercise

Switch to an embedding model with a different dimension count. Build a separate index, re-embed the corpus, compare evaluation results, and plan rollback instead of overwriting live vectors in place.

Make the learning stick.

Can equal-dimensional embeddings from different models be mixed?

Generally no. Equal shape does not imply a shared coordinate system or semantic geometry. Store model/version metadata and evaluate a deliberate migration rather than combining incompatible vectors.

Why does a vector index need metadata filtering?

Similarity alone does not enforce tenant boundaries, document type, language, time range, or permissions. Filtering can also change recall and performance, so test the actual filtered workload rather than assuming unfiltered benchmark results apply.

Is higher recall always worth the cost?

Not automatically. Evaluate whether additional candidates improve final answers or rankings under latency, memory, and cost constraints. A more expensive index setting may retrieve irrelevant near-duplicates without meaningful user benefit.

Avoid these traps

- Interpreting cosine similarity as a calibrated confidence probability.
- Comparing incompatible embedding versions directly.
- Benchmarking only unfiltered queries on an unrealistic corpus.

Ready for the next step?

- Similarity and normalization choices are explained.
- Approximation is evaluated against an exact baseline where feasible.
- Hybrid and reranking stages have measured value.
- Version migration, filtering, and deletion are operationally defined.

Reliable next reads

- [Google ML embeddings](#)
- [FAISS documentation](#)
- [Azure vector search overview](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.